

V3 Roadmap

1. Implement the Autonomous Curator Agent with a Live Market Data Feed

Build the Curator Agent as a fully autonomous, always-running background process that sources real market data without any human input and publishes insights on-chain every 30 minutes. This agent is the supply side of the marketplace. It connects to two free-tier data sources — Yahoo Finance via `yfinance` for NSE/BSE price and volume data, and NewsAPI (free tier, 100 requests/day) for market-moving headlines — synthesises them into a short, structured insight using Gemini, and calls the `/list` FastAPI endpoint with its own wallet to create a real on-chain listing. The Curator Agent must run as a background process alongside the FastAPI server, not as a manual script. Use Python's `APScheduler` library to trigger the fetch-synthesise-publish cycle every 30 minutes. Every published insight must be tagged with a `source_type: "curator_agent"` field in the ASA metadata so the Buyer Agent and frontend can distinguish Curator-generated listings from human-uploaded ones.

2. Build the Buyer Agent's Fully Autonomous Mode with Zero Human Keystrokes

Right now the Buyer Agent pauses and waits for the user to type "approve" before every `x402` payment. Build a proper `--autonomous` flag that removes this gate entirely when the configured trust thresholds are met, allowing the agent to complete an end-to-end purchase — search, evaluate, pay, unlock — with no keyboard input. The autonomous mode must use a strict, configurable, three-condition gate: `relevance_score >= AUTO_MIN_RELEVANCE AND reputation_score >= AUTO_MIN_REPUTATION AND price_usdc <= AUTO_MAX_PRICE`. If all three conditions pass, the agent proceeds directly to `x402` payment simulation and broadcast. If any condition fails, it skips the listing and explains why in the log. This is the live A2A transaction the judges described: one purchase cycle where an AI agent (Buyer) pays another AI agent (Curator) with zero human involvement.

3. Build the AgentRegistry Smart Contract and Identity Verification System

Every agent in Mercator — Buyer, Curator, and any future third-party agent — must have a verifiable on-chain identity before it can list or purchase insights. Build `AgentRegistry` as an ARC-4 smart contract that stores a registry of known agent wallets, their roles, and their signed identity manifests. The identity manifest is a JSON string `{"agent_name": "...", "wallet": "...", "role": "buyer|curator|seller", "registered_at_round": N}` signed with the agent's Algorand private key using `algosdk.util.sign_bytes`. Any contract that receives a transaction from an agent — `InsightListing`, `Escrow`, `SubscriptionManager` — reads the `AgentRegistry` to check `is_registered(wallet)` before proceeding. Unregistered wallets get a clean, readable revert: "Unregistered agent — call `AgentRegistry.register()` first".

4. Implement the FeeConfig Contract and Enforce the 2.5% Platform Fee On-Chain

The business model — 2.5% platform fee on every purchase — must be enforced in the smart contract, not in the backend. Any bypass of the backend should still result in the correct fee being deducted. Build FeeConfig as a standalone ARC-4 contract with an adjustable fee rate (stored in basis points, so 2.5% = 250) and a treasury wallet address. Modify Escrow.release_after_payment() so it calls FeeConfig.calculate_fee(amount) and constructs a three-output atomic transaction group: seller_payout = amount - fee, treasury_fee = fee, buyer_access_confirmed = true. All three transactions must be in the same atomic group — if any one fails, all three revert.

5. Build the SubscriptionManager Contract and Recurring Curator Feed

The subscription model is the clearest path from hackathon demo to real recurring revenue. Build SubscriptionManager as an ARC-4 contract that accepts a one-time USDC payment for a fixed number of Algorand rounds, records the subscriber's expiry round, and allows the Escrow contract to skip the per-insight payment check for active subscribers. A subscriber who pays 50 USDC receives access to all Curator Agent listings for 172,800 rounds (approximately 30 days at 5-second block time). The Buyer Agent checks SubscriptionManager.is_active(wallet) before every purchase — if active, it calls Escrow.release_for_subscriber() instead of triggering x402. This single contract changes Mercator from a transactional marketplace into a recurring-revenue SaaS product.

6. Build the Real-Time WebSocket Layer for Live Frontend Updates

Right now every frontend update requires a manual page refresh or a polling interval. Build a WebSocket connection from the FastAPI backend to the React frontend so the Discover Insights page updates in real time when a new Curator listing is published, the Activity Ledger updates instantly when a transaction is confirmed on-chain, and the conviction/agent status panels show live state. Use FastAPI's built-in WebSocket support and React's native useEffect + WebSocket API — no additional libraries required. Every significant backend event — new listing, payment confirmed, escrow released, reputation updated — broadcasts a JSON message to all connected clients with {event_type, timestamp, payload}.

7. Build the On-Chain Reputation Update Flow After Every Purchase

Right now the Reputation contract exists and can store scores, but it is never called automatically after a purchase completes. Wire the reputation update into the post-payment flow so it happens atomically with the escrow release — every successful purchase increases the seller's reputation score by a fixed amount (default: +5 points) and records the buyer's address as a verified purchaser. Over time, a seller's score reflects their actual sales history, not just manual updates. Add a reputation decay mechanism: if a seller

has no purchases in 30,000 rounds (~48 hours), their score decreases by 1 point per 10,000 rounds of inactivity, capped at a minimum of 0.

8. Build the FlowTracer and Downloadable Session Audit Log

Every significant event in the Mercator system — from the moment a query is entered to the moment the insight text is displayed — must be captured, timestamped, and exportable as a single JSON file that serves as standalone proof of the full system working. Build FlowTracer as a lightweight singleton that each module imports and calls without needing to pass it as a function argument. Each trace event has: `event_id` (UUID), `session_id`, `event_name`, `timestamp_iso`, `status` (success|failure|pending), `duration_ms`, `wallet_involved`, `tx_id` (nullable), `ipfs_cid` (nullable), `error_message` (nullable), and `plain_english_description`. At the end of every session, the tracer exports a `flow_trace_{session_id}.json` file and simultaneously inserts all events into a `flow_events` SQLite table.

9. Build the Custodial Onboarding Flow for Non-Crypto Users

A judge who is not a crypto native should be able to visit the staging URL, sign up with an email and password, get a funded Algorand wallet automatically, and complete a real TestNet purchase — all without ever seeing a seed phrase, installing a browser extension, or knowing what ALGO is. This is the "can a normal person use this" test the judges described. Build a complete custodial onboarding shell: server-side wallet generation, AES-256 encrypted mnemonic storage, automatic faucet funding on signup, and a seamless hand-off to the Buyer Agent flow after signup.

10. Build the Full API Access Layer for Developer Integrations

Mercator's B2B angle — charging fintech apps to embed the Buyer Agent as a paid API feature — requires a clean, documented, rate-limited API that a developer can integrate in one afternoon. Build MercatorAPI v1 as a separate FastAPI router mounted at `/api/v1/` with its own authentication, rate limiting, and response schema. Every endpoint must return consistent JSON with `{success, data, error, request_id, timestamp}` so developer integrations are predictable.

11. Rebuild the Semantic Search Tool with Re-Ranking and Diversity Logic

The current semantic search returns the top-3 results purely by cosine similarity + reputation score. This produces monotonous results when the Curator Agent has published many similar insights — every query returns three near-identical NIFTY breakout summaries. Add a Maximum Marginal Relevance (MMR) re-ranking step that penalises results which are too similar to each other, ensuring the top-3 results are both relevant and diverse. Also add a `min_reputation` filter so the agent never even considers listings from sellers below threshold — currently this check happens after retrieval, wasting Gemini embedding calls on listings that will be skipped anyway.

12. Add the Chain-of-Thought Evaluation Prompt v2 with Score Calibration

The current evaluation prompt gives the Judge a rubric but the scores cluster around 50/50 — the agent rarely makes a strong buy or skip decision because the prompt does not force enough differentiation between strong and weak insights. Rewrite the evaluation prompt to force a score distribution: the agent must assign a `buy_confidence` score from 0–100, and `trigger_x402_payment` only fires when `buy_confidence` ≥ 75 . Below 75, the agent explains exactly what would need to change to make the listing worth buying. This turns the evaluation step from a binary gate into a nuanced, logged decision that demonstrates the agent is genuinely reasoning about quality.

13. Implement Atomic Transaction Group Safety Across All Contract Interactions

Every money movement in Mercator — listing creation, escrow release, fee deduction, reputation update, subscription payment — must happen inside an atomic transaction group. If any one transaction in the group fails, all of them must revert. Right now, the escrow release and reputation update are separate, sequential transactions — a network failure between them leaves the escrow released but the reputation unchanged. Fix every multi-step transaction sequence to use `AtomicTransactionComposer` from `algosdk`.

14. Build the Operations Dashboard as a Real-Time Health Monitor

The Operations dashboard currently shows static config values. Upgrade it to a real-time health monitor that pulls live data every 10 seconds: current Algorand TestNet block height, all five contract states, IPFS gateway latency, backend endpoint response times, Curator Agent last run timestamp, WebSocket connection count, and total USDC volume processed on TestNet. Every metric must have a threshold: green (healthy), yellow (degraded), red (down). Any red metric automatically sends a broadcast via WebSocket so the frontend shows an alert banner on every page.

15. Build the Seller Reputation History and Public Seller Profile

Right now sellers are anonymous wallet addresses. Build a lightweight public seller profile that any Buyer Agent or human user can inspect before purchasing. A seller profile shows: wallet address (truncated), display name (optional, stored in `AgentRegistry`), reputation score with history chart, total insights sold, total USDC earned, average insight price, and a paginated list of past listings (with purchase counts). This turns the on-chain data into a human-readable trust signal and gives sellers an incentive to maintain quality.

16. Add Expiring Listings and Listing State Machine

Every insight has a shelf life. A NIFTY breakout call from 48 hours ago is worthless today. Add listing expiry to InsightListing so every listing has an expiry_round field (default: 172,800 rounds $\approx$ 48 hours). After expiry, InsightListing.get_active_listings() excludes expired listings from search results, and the Escrow.release_after_payment() method reverts if the listing is expired. Add a proper state machine to each listing: ACTIVE $\rightarrow$ SOLD (after first purchase) $\rightarrow$ EXPIRED (after expiry_round). Show the listing state visually in the InsightDetail page with a countdown timer.

17. Build the Full Error Path Coverage and Graceful Degradation System

Every function in Mercator can fail. IPFS can be down. The Algorand TestNet can lag. Gemini can rate-limit. x402 can reject. The Buyer Agent can produce malformed JSON. Each failure must produce: a structured log entry, a user-facing message in plain English, a FlowTracer failure event, and a recovery suggestion. Build a centralised ErrorHandler class that every module imports — instead of each module writing its own ad-hoc except block, every error flows through ErrorHandler.handle(error, context) which maps error types to user messages and log entries.

18. Build the GitHub Actions CI/CD Pipeline with Full Test and Deploy Automation

Every push to main must: run the full test suite, check coverage $\geq 90\%$, lint Python and TypeScript, and if all pass — deploy to Railway (backend) and Vercel (frontend) automatically. This pipeline is what transforms the repo from a hackathon project into something that looks like a real software product. The CI badge in the README is one of the first things a technical judge looks at.

19. Optimise the FastAPI Backend for Sub-Second Response Times

On the free Railway tier, the backend has limited CPU. Every endpoint must respond in under one second for a good demo experience. Profile every endpoint, identify the bottlenecks (most likely: synchronous Algorand indexer calls, blocking Gemini embedding calls, uncached IPFS fetches), and apply targeted fixes: async everywhere, connection pooling for the algod client, LRU caching for listing metadata, and pre-warmed Gemini embeddings.

20. Final TestNet Evidence Run and Submission Package Assembly

Do one complete, clean, from-scratch run of the entire Round 3 system on TestNet with fresh wallets and freshly deployed contracts. Every transaction from this run becomes part of the official submission evidence package. This run must demonstrate: Curator Agent autonomous publish, Buyer Agent A2A autonomous purchase, subscription purchase, seller reputation update, fee collection, agent identity

verification, and at least one clean failure handled gracefully. Nothing in the evidence package should be from a dev/test session — it must all be from this one clean run.

21. Write the One-Sentence Pitch and Lock It Across All Materials

The single most damaging thing in a 15-minute office hours slot is taking 90 seconds to explain what your product is. Every judge who has asked "what does Mercator do?" should hear the same sentence within 10 seconds — whether they are reading the README, watching the demo video, looking at your slide, or listening to your verbal pitch. Write the one sentence now, pressure-test it with five people who know nothing about crypto or AI, and lock it across every surface: README headline, GitHub repo description, Twitter bio, slide deck opener, and the first line of the demo video.

22. Conduct Three Customer Discovery Conversations Before the Office Hours Slot

The judges said: "Speak to them whether they want to use it, why they would, and if the product would really help them." Three real conversations with people in the target market is the minimum credible signal. One conversation is a coincidence. Three is a pattern. Target: one retail NSE options trader (individual), one small prop trading firm employee, one developer at a fintech startup. Ask the same five questions in every conversation. Write up the findings in VALIDATION.md before the submission.

23. Build the 10-Slide Pitch Deck for the 15-Minute Office Hours Slot

The office hours slot is 15 minutes: 5 minutes demo, 5 minutes Q&A on business model, 5 minutes on roadmap and ask. The pitch deck is not for reading during the demo — it is a leave-behind and a pre-read. Every slide must answer one specific judge question from the Twitter Space. No slide should have more than 30 words of text. Build the deck in Google Slides so it can be shared as a public link and embedded in the README.

24. Write BUSINESS_MODEL.md as a Standalone Document, Not a README Section

The judges said: "Have a business plan around it. Know pricing details." A one-paragraph business model buried at the bottom of the README will not survive a 15-minute office hours session where every judge is asking hard questions. Write BUSINESS_MODEL.md as a complete, standalone document that answers every revenue question before it is asked. Treat it as a simple investor memo — not a technical doc, not a slide deck, not a list of features.

25. Record the Demo Video with a Clear Narrative Arc

The demo video is a submission artefact. Judges watch it before deciding whether to shortlist. It must show the full system — not just a UI walkthrough — and every claim it makes must be verifiable on the Algorand TestNet explorer. Aim for 2.5 minutes. Not 4 minutes with padding. Not 1.5 minutes that skips the hard parts. 2.5 minutes with a clear narrative: problem → product → proof.

26. Maintain GitHub Hygiene as an Ongoing Daily Practice

GitHub hygiene is an explicit selection criterion. This is not a one-time cleanup — it is a daily discipline for the rest of the submission period. Every commit must be meaningful, every PR must describe what changed and why, the main branch must always be deployable, and the issue tracker must show real project management activity — not be empty. Judges who look at the commit history should see steady, professional progress, not a flurry of "fix fix fix" commits the night before submission.

27. Build the VALIDATION.md Document as Proof of Market Contact

The judges are not primarily evaluating technical complexity — they are evaluating whether the team has talked to the market and whether the product solves a real problem. VALIDATION.md is the document that proves you have had real conversations, received real feedback, and made real decisions based on what you heard. It is not a testimonials page. It is a structured research document.

28. Write OFFICE_HOURS.md with Pre-Prepared Answers to Every Likely Judge Question

You have 15 minutes. The judges will ask hard questions and expect sharp answers. "I haven't thought about that yet" in a 15-minute slot is a submission-ending response. Write OFFICE_HOURS.md now — before the slot — with a crisp, pre-prepared answer (maximum 60 seconds to say aloud) for every question a judge might plausibly ask based on the Twitter Space notes.

29. Write MAINNET_READINESS.md as a Concrete Technical + Legal Checklist

The judges said: "Path for mainnet. How can you be mainnet ready." This is not a vague roadmap item — it is a specific checklist of what changes when you flip from TestNet to Mainnet. Every difference between the current TestNet setup and a production Mainnet deployment must be documented. Judges reading this document should be able to see exactly how many steps remain and how hard each one is.

30. Assemble and Submit the Complete Round 3 Package by the Deadline

Everything built, written, recorded, and documented across all 30 steps above means nothing if it is not submitted correctly, completely, and on time. Treat the submission itself as a project task with subtasks,

not as a 10-minute form fill. Do the submission 24 hours before the deadline so you have buffer for portal errors, video upload failures, or last-minute fixes.